Atividade 02 – Criptografia RSA

1. Identificação

Nome: Pedro Henrique Vieira dos Santos

Matrícula: 2622130002

Disciplina: Estruturas Matemáticas para Computação

Turno: Matutino

2. Problema

A atividade consiste no desenvolvimento de um programa para trabalhar com o sistema de criptografia RSA, utilizando conceitos de números primos, aritmética modular e exponenciação modular.

O programa deve verificar a primalidade de dois números inteiros distintos, calcular os parâmetros necessários para a construção das chaves RSA, realizar a cifragem e a decifragem de uma mensagem inteira e demonstrar a aplicação dos Teoremas de Euler e de Fermat.

O objetivo é compreender o funcionamento matemático do RSA, demonstrando como são construídas as chaves e como uma mensagem pode ser cifrada e posteriormente recuperada, apresentando os cálculos realizados.

3. Procedimento matemático

Para desenvolver o programa, foram utilizados os conceitos de primalidade, máximo divisor comum, função totiente de Euler, inverso multiplicativo e exponenciação modular.

Primalidade e parâmetros RSA

Inicialmente, são escolhidos dois números primos distintos, representados por p e q.

A partir deles, são calculados:

$n = p \times q$

$\phi(n) = (p - 1) \times (q - 1)$

O valor de n é utilizado como módulo nas operações de cifragem e decifragem, enquanto $\phi(n)$ é utilizado na construção das chaves.

Construção das chaves

O expoente público e deve ser maior que 1, menor que $\phi(n)$ e coprimo com ele, ou seja:

$MDC(e, \phi(n)) = 1$

Em seguida, calcula-se o expoente privado d, que corresponde ao inverso multiplicativo de e módulo $\phi(n)$.

A relação matemática utilizada é:

$e \times d \equiv 1 \pmod{\phi(n)}$

Assim, são construídas as chaves:

- Chave pública: (e, n).
- Chave privada: (d, n).

O inverso multiplicativo é calculado utilizando o algoritmo de Euclides estendido.

Cifragem e decifragem

Para realizar a cifragem, o programa recebe uma mensagem inteira M, respeitando a condição:

$0 \leq M < n$

A mensagem é transformada em um número cifrado C utilizando a chave pública:

$C \equiv M^e \pmod{n}$

Para recuperar a mensagem original, utiliza-se a chave privada:

$M' \equiv C^d \pmod{n}$

Ao final, o programa compara a mensagem recuperada com a mensagem original, verificando se $M' = M$.

Exponenciação modular

Para calcular as potências utilizadas na cifragem e na decifragem, foi implementado o método de exponenciação modular por reduções sucessivas.

Esse procedimento realiza multiplicações e elevações ao quadrado, aplicando o módulo durante os cálculos. Dessa forma, evita a construção de números inteiros muito grandes e torna o processo mais eficiente.

Teorema de Euler

O Teorema de Euler estabelece que, quando uma base `M` é coprima com o módulo `n`, temos:

`M^ϕ(n) ≡ 1 (mod n)`

A condição necessária para sua aplicação é:

`MDC(M, n) = 1`

O programa verifica essa condição antes de demonstrar o teorema.

Pequeno Teorema de Fermat

O Pequeno Teorema de Fermat estabelece que, para um módulo primo `p` e uma base `a` que não seja divisível por `p`, temos:

`a^(p-1) ≡ 1 (mod p)`

O programa utiliza o número primo `p` escolhido na construção das chaves e solicita uma base `a` que não seja divisível por ele.

É importante destacar que o Pequeno Teorema de Fermat não é aplicado diretamente ao módulo RSA `n`, pois ele é formado pelo produto de dois números primos distintos e, portanto, é composto.

4. Implementação

O programa foi desenvolvido em Python, utilizando o arquivo:

`rsa.py`

Foram implementadas funções próprias para verificar se um número é primo, calcular o máximo divisor comum, encontrar o inverso multiplicativo e realizar a exponenciação modular.

Inicialmente, o programa recebe os números p e q e verifica se são primos, distintos e maiores que 1. Depois, calcula o módulo RSA e a função totiente de Euler.

Na construção das chaves, o programa recebe o expoente público e, verifica suas condições matemáticas e calcula o expoente privado d por meio do algoritmo de Euclides estendido.

Na etapa seguinte, recebe uma mensagem inteira, realiza sua cifragem utilizando a chave pública e sua decifragem utilizando a chave privada. Os resultados são apresentados na tela, juntamente com as etapas da exponenciação modular.

Por fim, o programa demonstra os Teoremas de Euler e Fermat, verificando as condições necessárias para a aplicação de cada um.

Os procedimentos matemáticos centrais foram implementados manualmente, utilizando estruturas de repetição, condições e operações aritméticas da linguagem Python.

5. Testes

Foram utilizados diferentes valores para verificar o funcionamento do programa, incluindo situações válidas e entradas que não atendem às condições matemáticas exigidas.

Parte A – Primalidade e parâmetros RSA

Teste 1 – Números primos válidos

- `p = 5`
- `q = 11`

Cálculos:

$n = 5 \times 11 = 55$

$\phi(n) = (5 - 1) \times (11 - 1)$

$\phi(n) = 4 \times 10 = 40$

Resultado: os números foram aceitos e os parâmetros RSA foram calculados corretamente.

Teste 2 – Números inválidos

Foram testados valores como números compostos, números menores que 2 e números primos iguais.

Resultado: o programa identifica as entradas inválidas e solicita novos valores.

Parte B – Construção das chaves

Teste 1 – Expoente público válido

- `e = 3`
- `ϕ(n) = 40`

Verificação:

`MDC(3, 40) = 1`

Cálculo do inverso:

`3 × 27 = 81`

`81 % 40 = 1`

Portanto:

`d = 27`

Resultado:

- Chave pública: `(3, 55)`.
- Chave privada: `(27, 55)`.

Teste 2 – Expoente público inválido

Foi utilizado um valor de e que não atende às condições exigidas, como `e = 2`.

Resultado: o programa rejeita o valor, pois `MDC(2, 40) ≠ 1`, e solicita outro expoente.

Parte C – Cifragem e decifragem

Teste 1 – Mensagem válida

- `M = 7`

- $e = 3$
- $d = 27$
- $n = 55$

Cifragem:

$C \equiv 7^3 \pmod{55}$

$C \equiv 343 \pmod{55}$

$C = 13$

Decifragem:

$M' \equiv 13^{27} \pmod{55}$

$M' = 7$

Resultado: a mensagem original foi recuperada corretamente, pois $M' = M$.

Teste 2 – Mensagem fora do intervalo

Foi informado um valor de mensagem igual ou superior a n, ou um valor negativo.

Resultado: o programa rejeita a entrada e solicita uma mensagem que respeite a condição $0 \leq M < n$.

Parte D – Teoremas de Euler e Fermat

Teste 1 – Teorema de Euler

Utilizando:

- $M = 7$
- $n = 55$
- $\phi(n) = 40$

Verificação da condição:

$MDC(7, 55) = 1$

Aplicação do teorema:

$7^{40} \bmod 55 = 1$

Resultado: a condição de coprimalidade foi satisfeita e o Teorema de Euler foi confirmado.

Teste 2 – Base não coprima com o módulo

Utilizando `M = 5` e `n = 55`:

`MDC(5, 55) = 5`

Resultado: o programa identifica que a base não é coprima com o módulo e informa que não é possível aplicar diretamente o Teorema de Euler.

Teste 3 – Pequeno Teorema de Fermat

Utilizando:

- `a = 2`
- `p = 5`

Cálculo:

`2^(5-1) mod 5 = 2⁴ mod 5`

`16 mod 5 = 1`

Resultado: as condições foram satisfeitas e o Pequeno Teorema de Fermat foi confirmado.

Teste 4 – Base divisível pelo módulo primo

Foi informada uma base igual a 5, considerando `p = 5`.

Resultado: o programa rejeita a base e solicita outro valor, pois ela é divisível pelo módulo primo.

Conclusão dos testes

Os testes demonstraram o funcionamento dos procedimentos de construção das chaves RSA, cifragem e decifragem, além da aplicação dos Teoremas de Euler e Fermat. Também foram verificadas situações em que as condições matemáticas não são satisfeitas, garantindo que o programa apresente as validações correspondentes.